Shared Household Journal — Product Requirements Document

Claude Code in VS Code Edition | Repository-first implementation guide | Draft v2

Document status

Draft v2. This edition is written for development with Claude Code operating inside Visual Studio Code. It is a product requirements document, not permission to rewrite an existing repository. If an implementation already exists, inspect it and preserve working behavior unless a requirement explicitly calls for a change.

1. Product vision

Shared Household Journal helps roommates and shared households coordinate everyday money and home responsibilities without making the experience feel like accounting software. It combines expense tracking, clear balances, settlement records, a household activity journal, and practical shared-home tools in one calm, friendly workspace.

2. Product principles

• Household-first: a household is the main collaboration boundary.

• Explainable money: every balance can be traced to expenses, split rules, and recorded settlements.

• Human-centered: use warm, home-oriented language and avoid corporate finance jargon.

• Trustworthy: never imply that a payment was transferred when the app only records a settlement.

• Incremental delivery: build and verify small vertical slices; do not attempt a full rewrite in one prompt.

• Existing repository wins: source code, current conventions, and verified behavior take precedence over assumptions in this document.

3. Target users and jobs

Roommates, partners, students, families, and other people sharing a home. They need to record shared costs, understand who owes whom, settle up, remember recurring bills, coordinate chores and shopping, and maintain a transparent record of household activity.

4. Scope

Core: authentication and profiles; household creation and membership/invitations; expenses with equal, percentage, and custom splits; participant-level balances; settlement recording; journal/activity feed; recurring bill reminders; shopping list; chores; budgets and summaries; notification preferences; responsive interface; household-scoped access control.

Optional/later: receipt OCR, AI-assisted categorization, debt simplification, export, advanced analytics. These must be explicitly staged and must not block core expense correctness.

Out of scope unless separately approved: subscription billing or Stripe subscriptions; automatic bank transfers; claims of real payment processing; public household discovery; unapproved external sharing of private household data.

5. User journeys

A. Create account/sign in → create or join a household → invite members.

B. Record expense → choose payer, amount, date/category, participants, and split method → review allocation → save → balances update.

C. Review balances → inspect underlying transactions → record a settlement → balances update without representing an external transfer.

D. Add recurring bill → choose cadence and reminder → upcoming item appears in household context.

E. Use journal, shopping list, and chores to coordinate home life.

F. View household summaries and budgets with clear time ranges and explanations.

6. Functional requirements

FR-01 Authentication: users can register, sign in, sign out, and manage basic profile details using the existing auth provider if already configured.

FR-02 Household: authorized users can create a household, join via a secure invite mechanism, view members, and leave where policy permits.

FR-03 Expenses: create, view, edit, and delete expenses subject to role and household access rules.

FR-04 Splits: support equal, percentage, and custom-amount splits. Validate that allocations sum to the expense total within a defined currency precision.

FR-05 Balances: derive balances from recorded expenses and settlements; avoid storing a mutable balance as the sole source of truth.

FR-06 Settlements: record payer, recipient, amount, date, and optional note. Label as recorded/settled in-app; do not imply money moved externally.

FR-07 Journal: show meaningful household activity with timestamps and actor attribution where available.

FR-08 Recurring bills: store recurrence configuration and upcoming due information; prevent accidental duplicate generation.

FR-09 Shared tools: provide shopping list and chores with completion/assignment state.

FR-10 Analytics/budgets: display understandable totals and trends; calculations must be reproducible from underlying records.

FR-11 Responsive UX: core flows work on mobile, tablet, and desktop.

FR-12 Access control: all household data is restricted to authorized household members.

FR-13 Errors/loading/empty states: every primary flow has useful states and recovery guidance.

FR-14 Accessibility: semantic controls, keyboard navigation, visible focus, labels, and sufficient contrast.

7. Product design direction

Warm, practical, and uncluttered; a shared-home journal rather than a corporate finance dashboard. Palette: primary #4F7CAC, secondary #7FB069, accent #F4A259, background #FAF8F5, cards #FFFFFF, text #2D3748, success #22C55E, warning #F59E0B, danger #EF4444. Suggested typography: Plus Jakarta Sans or Inter. Existing design tokens and installed UI conventions should be audited before adding or replacing libraries.

8. Acceptance criteria

• A new expense is persisted and visible to authorized household members.

• Split validation rejects invalid allocations and handles currency precision consistently.

• Balance results match independently calculated test cases, including expenses and settlements.

• A user from another household cannot read or mutate household records through UI or direct API/database access.

• Settlement wording is accurate and never claims an external payment occurred.

• Primary pages have loading, empty, error, and success states.

• Mobile layouts remain usable without horizontal overflow in the supported viewport range.

• Existing working features remain functional after each phase.

• Test claims in release notes are based on commands actually run; failures and unrun checks are disclosed.

9. Delivery phases

Phase 0 — Repository discovery and baseline: inspect files, git status, package scripts, architecture, environment variable names, existing tests, and current app behavior. No feature changes until the audit is summarized.

Phase 1 — UI foundation: establish or reconcile tokens, typography, layout primitives, navigation, responsive shell, and accessible states while preserving the current framework and component system.

Phase 2 — Auth and household: account flow, household lifecycle, membership, invite flow, authorization.

Phase 3 — Expenses and splits: expense CRUD, validation, split engine, transaction tests.

Phase 4 — Balances and settlements: derived balances, settlement records, explainability, edge-case tests.

Phase 5 — Journal and notifications: activity events, notification preferences, in-app alerts.

Phase 6 — Recurring bills: cadence, due dates, reminders, duplicate protection.

Phase 7 — Shopping and chores: household lists, assignments, completion, activity history.

Phase 8 — Analytics and budgets: date filters, summaries, budget progress, reconciliation tests.

Phase 9 — Optional intelligence: OCR/categorization only after explicit approval, with manual correction and graceful fallback.

Phase 10 — Hardening and release: security review, RLS verification, accessibility, performance, deployment documentation, and final regression.

10. Claude Code operating rules

Claude Code must begin each phase by auditing the relevant repository areas and reporting what exists, what works, and what is missing. It must propose a short plan before editing. It must preserve working code and avoid broad rewrites, unnecessary dependencies, fabricated files, or speculative architecture. It must ask before destructive migrations, data deletion, replacing a major library, changing auth providers, or altering established APIs. It must not expose secrets or print .env values. It must not claim tests passed unless it ran them and observed success. After edits, it must report changed files, implementation decisions, commands actually run, results, known limitations, and the recommended next step.

11. Open decisions

Confirm with the product owner before implementation if not already decided: currency/multi-currency policy; household roles and invite expiry; whether deleted expenses are hard-deleted or soft-deleted; settlement reversal behavior; timezone policy for recurring bills; notification channels; data export/retention; OCR provider; and whether any advanced feature is included in the first release.